

Scaling Compute at Inference

Beyond Thinking Budget Scaling: Alternative Ways of Inference Compute Scaling

We explore an alternative axis for scaling reasoning models beyond simply extending their reasoning length. In contemporary CoT-style systems (e.g., QwQ, R1), the hidden <think> phase substantially increases time-to-first-token and inflates KV-cache size, leading to high latency. Our goal is to enable *compute scaling* during the reasoning phase, complementing length scaling. (1) **Scaling FFN compute**: we train Mixture-of-Experts (MoE) models where the number of active experts can be varied dynamically at inference. This allows compute to be adjusted at different granularities (e.g., per-token or per-sentence) without incurring overhead, while maintaining a single model and a single KV cache—avoiding the inefficiencies of draft→target pipelines. Through top-k cycling during training ([4, 8, 16,...]), the router learns to operate seamlessly across compute regimes, enabling adaptive inference that can be tuned to problem hardness or latency budgets. (2) **Scaling attention compute**: in parallel, we are investigating adaptive attention through variants such as 2-simplicial attention, with the aim of leveraging underutilized FLOPs during the memory-bound decoding phase.

Initial results for FFN scaling on Qwen3-30B-A3B and Flash-2.8B are promising. On light reasoning tasks such as GSM8K, accuracy is stable for $k \geq 4$ (e.g., $k=4 \rightarrow 91.5\%$, $k=8 \rightarrow 94.9\%$, $k=12 \rightarrow 95.5\%$). On math-reasoning tasks with explicit thinking budgets, performance scales with both the number of experts and available tokens: at 16k tokens, $k=4 \rightarrow 60\%$, $k=8 \rightarrow 56.7\%$, $k=16 \rightarrow 66.7\%$ (plateauing beyond 16 experts). Complementary finetuning of the internal **Flash-2.8B MoE** model showed gains of upto **~10%** on math and coding benchmarks when scaling from $k=4$ to $k=8$, with only minimal latency overhead. Together, these results suggest that adaptive compute allocation offers a practical new axis for scaling reasoning models.

Main Results

Model: 2.8B Flash MoE model

Training Dataset: Catalyst SFT Dataset

Baseline: training model with standard number of active experts (ie 4)

Our Model: trained to support different number of active experts (ie 4, 8, 12 and 16)

Pass@K accuracy after compute scaling for different length budgets

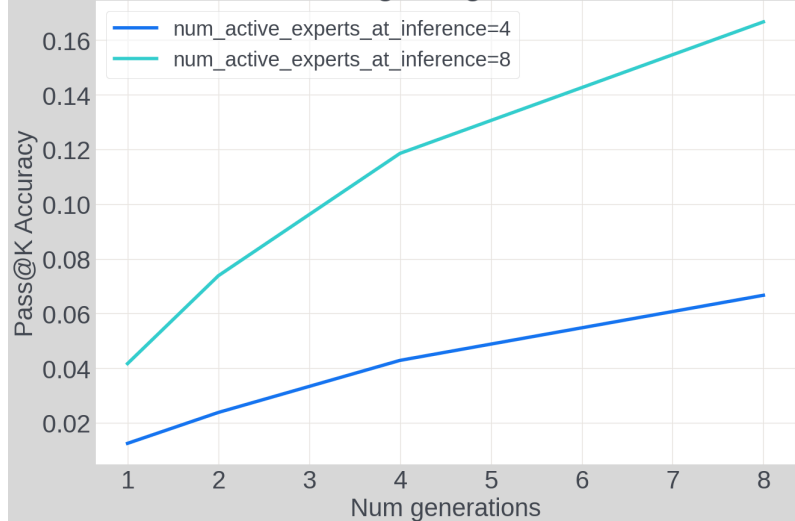
Math500 Level 4,5



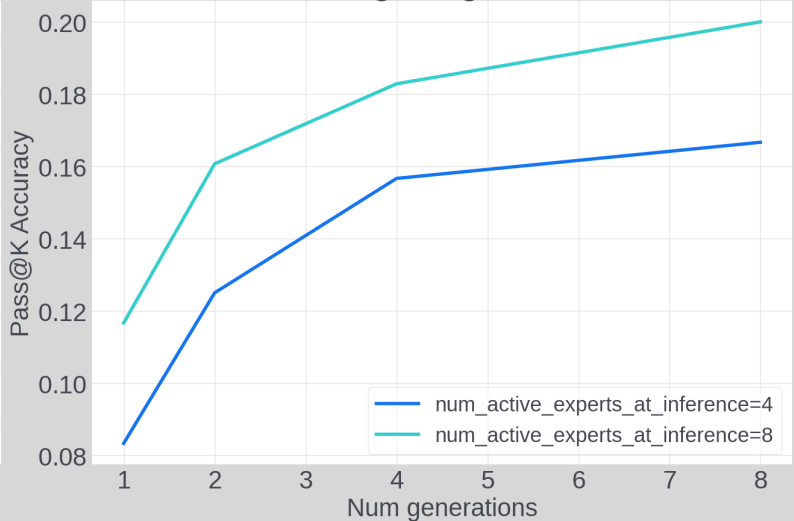
Note: num_active_experts_at_inference=4 results are from the baseline

AIME 2024

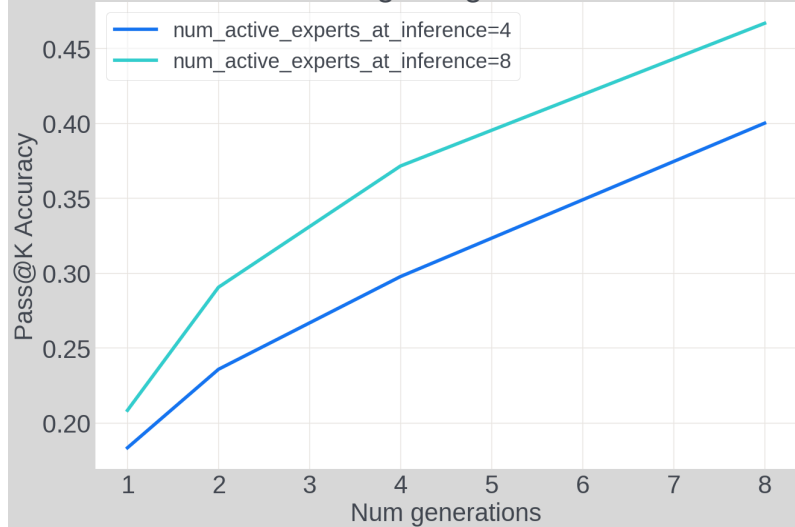
Thinking Budget = 2048



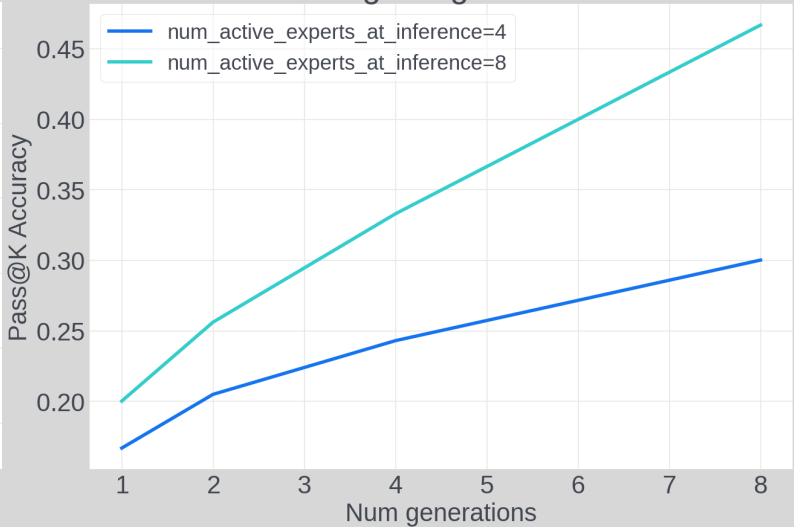
Thinking Budget = 4096



Thinking Budget = 8192

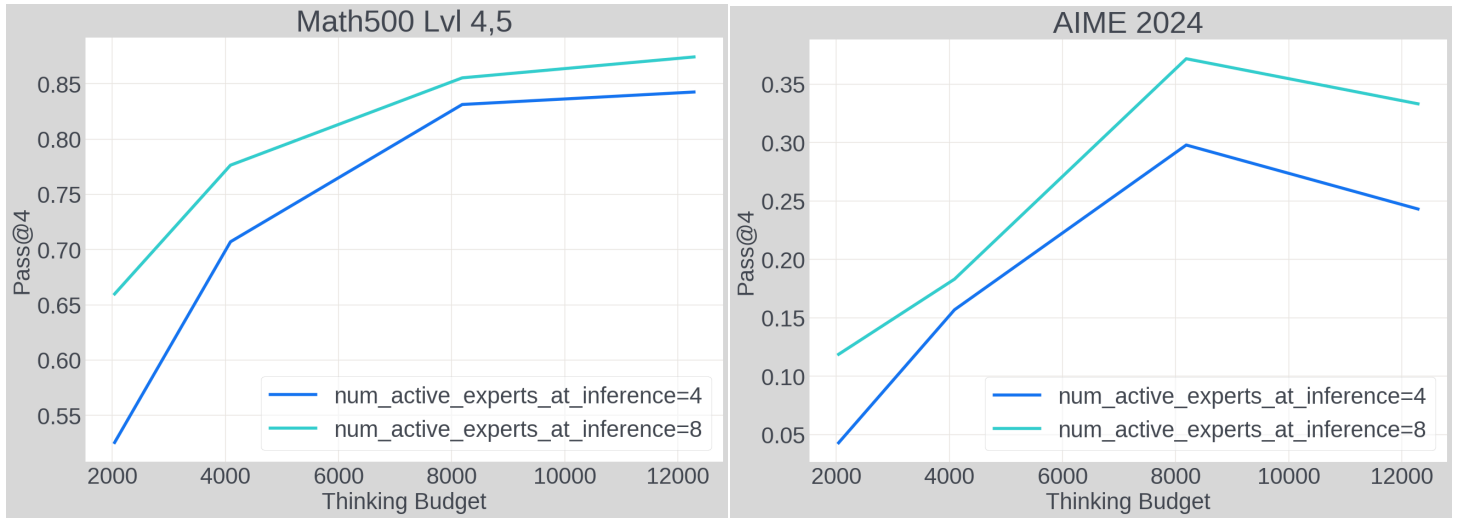


Thinking Budget = 12288



Note: num_active_experts_at_inference=4 results are from the baseline

Pass@4 accuracy for different length budgets



Note: num_active_experts_at_inference=4 results are from the baseline

Thoughts:

The performance increase seems to diminish with longer length budget, it might be because % of total compute increase due to scaling ffn decreases with context length because of increasing contribution from attention layers.

Thus we believe scaling attention compute (eg via triplet attention) should give even better gains at higher thinking budgets.

Running Notes

Training on Open Source Qwen3 Model

Model Chosen: **Qwen 30B A3B**, default number of active expert 8 out of 128

Dataset Chosen: Openthoughts3

1.3 Million trajectories from QwQ32B model for science, math and coding

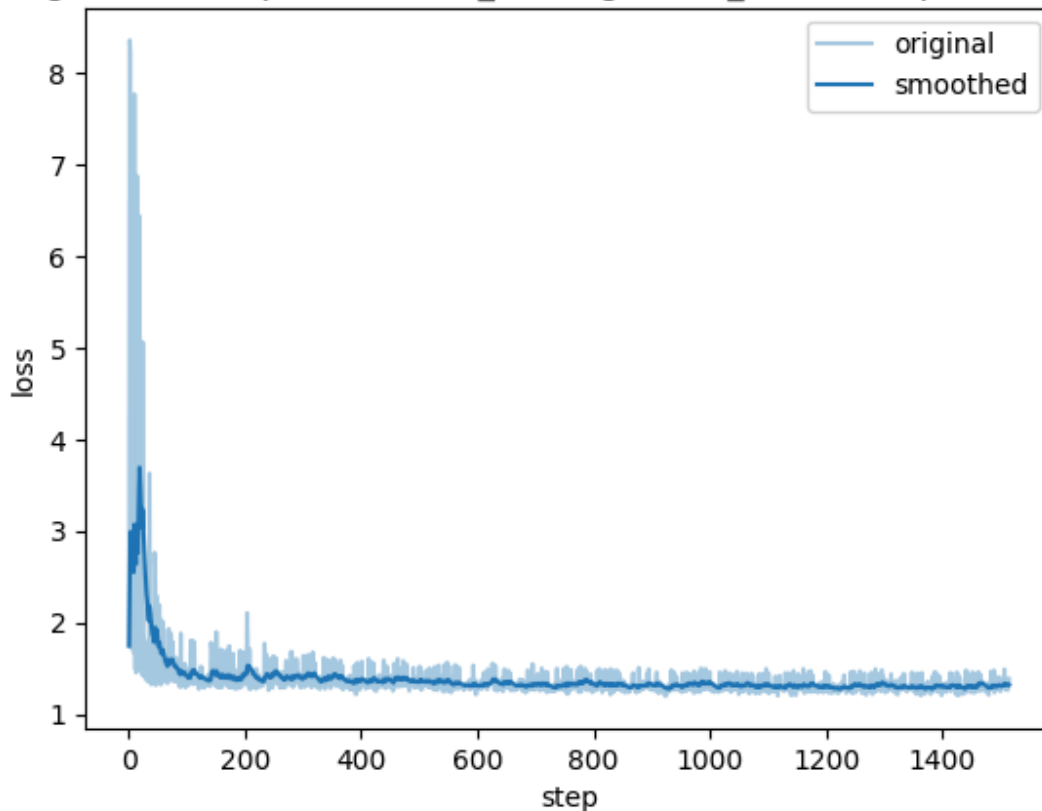
Checkpoint Choices:

- ☒ SFT of the reasoning model: ~~Done~~
- ☐ Take base model and post-train it: TODO

Training Details:

During the training cycle through active experts from [8, 12, 16, 20, 24, 32] each iteration. Note that I have drastically increased the number of active experts to see how much scaling is possible. I stopped training after 1.5B tokens after seeing the curve.

training loss of ../experiments/01_training/saved_models2/OpenThinker3-30



Issue: drastic increase in the loss during start of the training, due to this the original model weights are disturbed.

Possible Explanation:

- At the start of training using num_active expert directly around 32 will drastically change the distribution of activations thus huge gradients.

Possible Solutions:

- ☐ First keep the training space restricted to [8, 12], then after few iterations increase it to [8, 12, 16] and then to [8, 12, 16, 32]
- ☐ Keep the num_active_expert list [8, 12, 16, 32] but keep an additional check that low probable experts (whose probability is some threshold lower than 8th expert is not included) and then relaxing the threshold as the training progresses.

Few Results w/ this experiment

Dataset	Num Experts	Original	After Training
AIME 2025	8	43.3	26.7
	16	30.0	33.3
Math 500	8	95.0	89.6
	16	94.2	92.2

SFT on Base Model

Training Details:

First keep the training space restricted to [8, 12], then after few iterations increase it to [8, 12, 16] and then to [8, 12, 16, 20]

Note: The loss spike problem is fixed.

Context Length = 16k

16k * 1.2 million examples

Dataset	Num Experts	Original	SFT (k=8)	SFT (flexible)
AIME 2025	8	10	43.3	46.7
	16	10	23.3	53.3
	20	-	-	53.3
Math 500	8	76.2	92.6	91.2
	16	64.4	85.2	92.6
	20	-	-	93.4

Dataset	Budget	Num Experts	Original	SFT (k=8)	SFT (flexible)
	4k	8	10	26.7	30.0
	4k	12	10	30.0	36.7
	4k	16	-	26.7	40.0
	4k	20	-	16.7	33.3
	8k	8	-	33.3	36.7
	8k	12	-	36.7	40.0
AIME 2025	8k	16	-	30.0	46.7
	8k	20	-	20.0	43.3
	16k	8		40.0	46.7
	16k	12			43.3
	16k	16		33.3	53.3
	16k	20			53.3
	4k	8	76.2	92.0	91.2
	4k	12	64.4	87.8	92.0
	4k	16	-		92.2
	4k	20	-	81.6	91.6
	8k	8	-	94.4	93.8
MATH500	8k	12	-	92.4	94.0
	8k	16	-	87.6	93.6
	8k	20	-	83.2	

	16k	8		94.6	94.2
	16k	12			93.8
	16k	16		89.9	94.6
	16k	20			95.2

Base model:

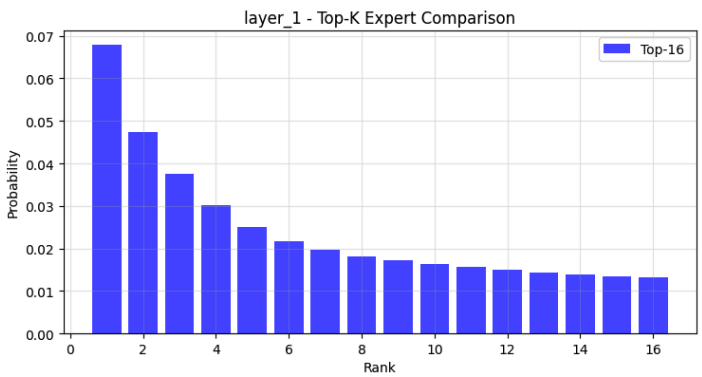
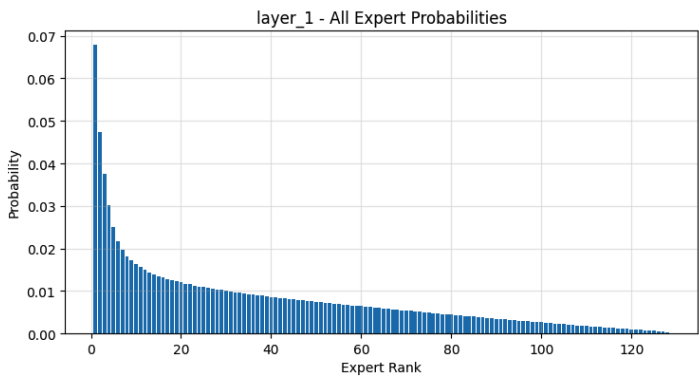
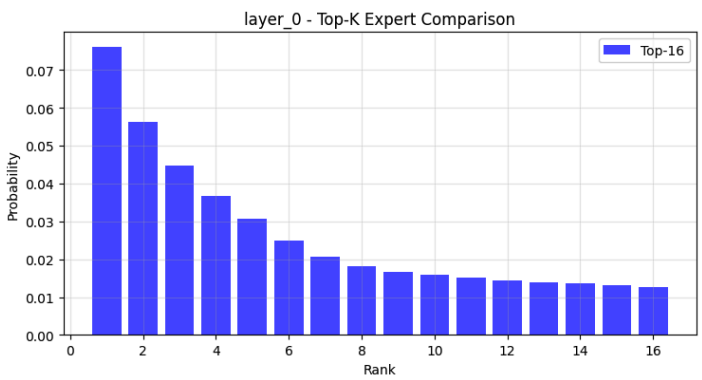
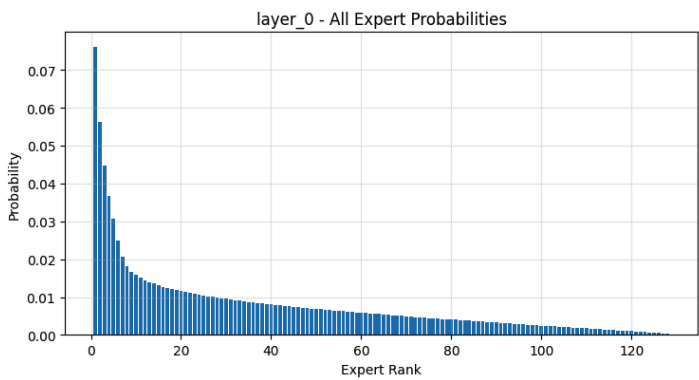
16k

4k -> 32k

Dataset:

Acc vs inference time:

Router Probabilities



Want to try next:

- ☐ Per token dynamic selection of experts based on probability.
- ☐ Analysing in what kind of problems does increasing the experts help more, does this have any connection with overthinking/underthinking.
- ☐ Check training with num_experts based on $\text{sum_probability} \leq x * p$, where p is sum of probabilities of top 8 experts and x is sampled randomly from [0.8 - 1.2]

pass@8, avg@8

Dynamic during inference

Estimation of inference cost

Dynamic Inference

k=8 : 33.3

k=12: 40.0

k=16: 46.7

Dynamic:

- $p > 0.016$ (avg k ~11): 0.40
- $p > 0.014$ (avg k ~14): 0.46

Effect of Temperature on Results

Budget	Num Experts	SFT (greedy decoding)	SFT (temp=0.6)	SFT pass@8 (temp=0.6)	SFT-flexible (greedy)	SFT-flexible (temp=0.6)	SFT-flex pass@8 (temp=0.6)
4k	8	26.7	36.6	53.3	26.7	36.6	53.3
4k	12				36.7	46.6	60.0
4k	16				40.0	33.3	53.3
8k	8	33.3	40.0	70.0	36.7	40.0	63.3
8k	12				40.0	50.0	70.0
8k	16				46.7	43.3	60.0
16k	8	40.0	66.66	80.0	46.7	53.3	73.3
16k	12				43.3	50.0	70.0
16k	16				53.3	53.3	80.0

OpenMath

Decreasing the temperature, temp=1, increase temperature > 1

Baseline with k=16

Stable Results

Greedy Decoding, pass@1 averaged over 8 runs

Budget	Active Experts	SFT Baseline (pass @1)	SFT flexible (pass @1)
4k	8	31.25±2.85	33.75±3.88
	12	33.33±3.33	34.98±2.37
	16	28.76±2.29	33.32±2.36
8k	8	42.91±6.95	37.51±4.31
	12	36.68±1.67	45.82±3.23
	16	30.40±3.08	36.65±3.73
16k	8	48.32±3.72	47.92±2.84
	12	40.00±4.08	64.18±4.66
	16	26.25±2.61	45.43±2.33

Temperature Sampling

Bud get	K	SFT Baseline T = (0.0)	SFT Baseline T = (0.6)	SFT flexible (T=0.0)	SFT flexible T = (0.4)	SFT flexible T = (0.6)	SFT flexible (T=1.0)
4k	8	31.25±2.85	32.51±4.64	33.34±3.3 4	33.76±5.11	32.91±3.5 1	32.90±3.0 9
	12	33.34±3.34	31.67±4.70	36.65±4.0 8	33.33±3.3 4	35.41±4.3 9	33.75±4.2 3
	16	28.76±2.30	27.94±5.25	34.17±3.2 3	36.67±4.7 3	33.33±3.3 4	32.50±5.7 0
8k	8	42.91±6.95	42.51±5.70	37.09±5.1 0	42.08±4.0 4	42.10±3.7 1	40.00±3.3 0
	12	36.69±1.68	37.09±5.13	50.40±4.5 3	43.34±6.2 4	43.76±5.11	39.56±4.2 2
	16	30.40±3.09	32.49±3.62	36.25±2.6 2	41.25±3.2 9	43.34±4.7 1	43.33±4.4 0
16k	8	48.33±3.72	58.33±3.72	45.84±4.6 6	56.24±4.5 4	52.09±5.2 6	52.91±4.2 2
	12	40.00±4.08	50.41±7.15	65.45±3.3 1	53.33±5.5 3	55.01±7.0 7	58.34±4.4 2
	16	26.25±2.62	37.50±7.01	44.58±1.6 5	53.34±4.0 8	54.59±8.3 4	51.66±4.0 7

34.06±3.80

42.71±4.75

Reasoning MOEs

Reasoning MOEs to speed up Reasoning Phase

Reasoning Models:

- Current Reasoning like QwQ has a reasoning phase and a separate answer phase, the reasoning phase is bounded by <think> tokens. Answer phase response is very similar to non-reasoning models, commonly a step-by-step solution.

Example-

Question: xxx

LLM Response:

<think> yyyyyyyyyyyyyyyyyyy

yyyyyyyyyyyyyyyyyy </think>

Lets solve this step by step:

Step_1: zzz

Step_2: zzz, hence the answer is \boxed{a}.

Challenges:

- **Time to First Token (TTFT):** Users care about the step-by-step solution and not the very lengthy reasoning process behind it and therefore the thinking phase is usually kept hidden, as a result the time to first token is increased considerably and can go up to orders of minutes and even hours.
- **Latency:** Since reasoning trajectories are usually very long the KV cache starts to grow by a lot very quickly thus hurting the latency of response.

Initial Findings

- Some initial findings done by independent groups at Berkeley suggests that the reasoning phase can be carried out by a weaker/smaller model and fed to the target model to produce the solution phase; this does not affect the performance by much.
- This offers a much better performance-latency tradeoff, in other words for a given time budget it is better to generate long but fast responses for the thinking phase and invest more time towards the solution phase.

Naive Approach (currently being tried by different groups)

- Use small draft model to produce reasoning trajectories and then feed them to the bigger target model to produce solution trajectories.

- Problem with this approach:
 - The KV cache of the small model cannot be reused by the target model and therefore the reasoning phase needs to be encoded again by the target model which takes up a long time as they are usually very long.
 - Storing 2 models, in large scale inference systems managing two models can be cumbersome similar to speculative decoding.

Proposal:

- During training fix few experts for reasoning phase
- Training should be done in a manner to allow changing the number of experts being used for reasoning phase depending on the hardness of problem and latency constraint.
- This approach offers following advantages:
 - No need to manage separate KV cache
 - No need to have extra prefill with target model
 - Dynamic time usage and reasoning strength based on the hardness of query.
 - No routing needed for the reasoning phase.

Side Explorations

1. Does performance keeps on increasing with scaling (length of reasoning)
 - a. I suppose not given model performance drops down for long contexts
 - b. If it drops, then is it helpful to have a cleaning phase to filter out useless reasoning parts.
2. Can we do some kind of streaming where the model starts outputting the solution while the reasoning is still going on in the background.

Discussions w/ Rishabh Agarwal

More generalized form of the idea:

- Use any adaptive inference compute network method eg Matformers or Looping transformers
- Show that you can add an axis of inference scaling (apart from length) by changing the adaptive compute in reasoning, we can give this control to user
- We can explore how much to divide this compute around in the thinking vs solution phase.

Timeline

Week 1 June 1 - June 7

- ☒ ~~Setup Flame MoE~~
- ☒ ~~Setup Qwen 3 MoE~~
- ☒ ~~Benchmark Qwen 3 MoE for different top-K.~~

Blockers:

- ☒ ~~Download speed of AWS cluster is very slow~~

Week 2 June 8 - June 14

- ☐ Finetune Qwen 3 MoE to use different number of experts.

Week 3&4 June 14 - June 27

- ☒ ~~Complete finetuning of Qwen 3 with different MoEs~~
- ☒ ~~Evaluate on Math500 and AIME 2024 to check how the performance scales with top_k~~

Experiments

Baselines:

Top 8: 0.816 (math 500 - cons@16), 0.3

Top 4: 0.832, 0.26

Dynamic:

Top 8: 0.824, 0.3

Top 4: 0.824, 0.2

Try top_k 16

Add logging for router weights

AIME 2025

Look for pass_at_1, pass_at_16

Get expectation for 4->8, using 1->2->4

Training Top K	Eval Top K	Math 500 lev 4,5 (cons@16)	Aime 2024 (cons@16)
8	8	0.816	0.3
4	4	0.832	0.26
dynamic	8	0.824	0.3
dynamic	4	0.824	0.2

Updates

- Wrote dynamic top_k training code on mast
- Trained baselines with top 2, 4, 8
- Trained dynamic top_k model
- Had to rerun the experiments as there were some bugs that were actively being fixed
- Training Details:

- Much larger scale than before ~2B tokens (can increase it further depending on results)

Blockers:

- Main evals were broken
- Skipped the evals while training
- Evals still not working with consolidated checkpoints

New Experimental Results:

1. Training starting with base model

At iteration 1000 -

top_k: 4, pass@4: 0.36, cost: 81.7537186299904

top_k: 8, pass@4: 0.56, cost: 113.4013925916672

top_k: 16, pass@4: 0.46, cost: 162.0092435185664

At iteration 5000 -

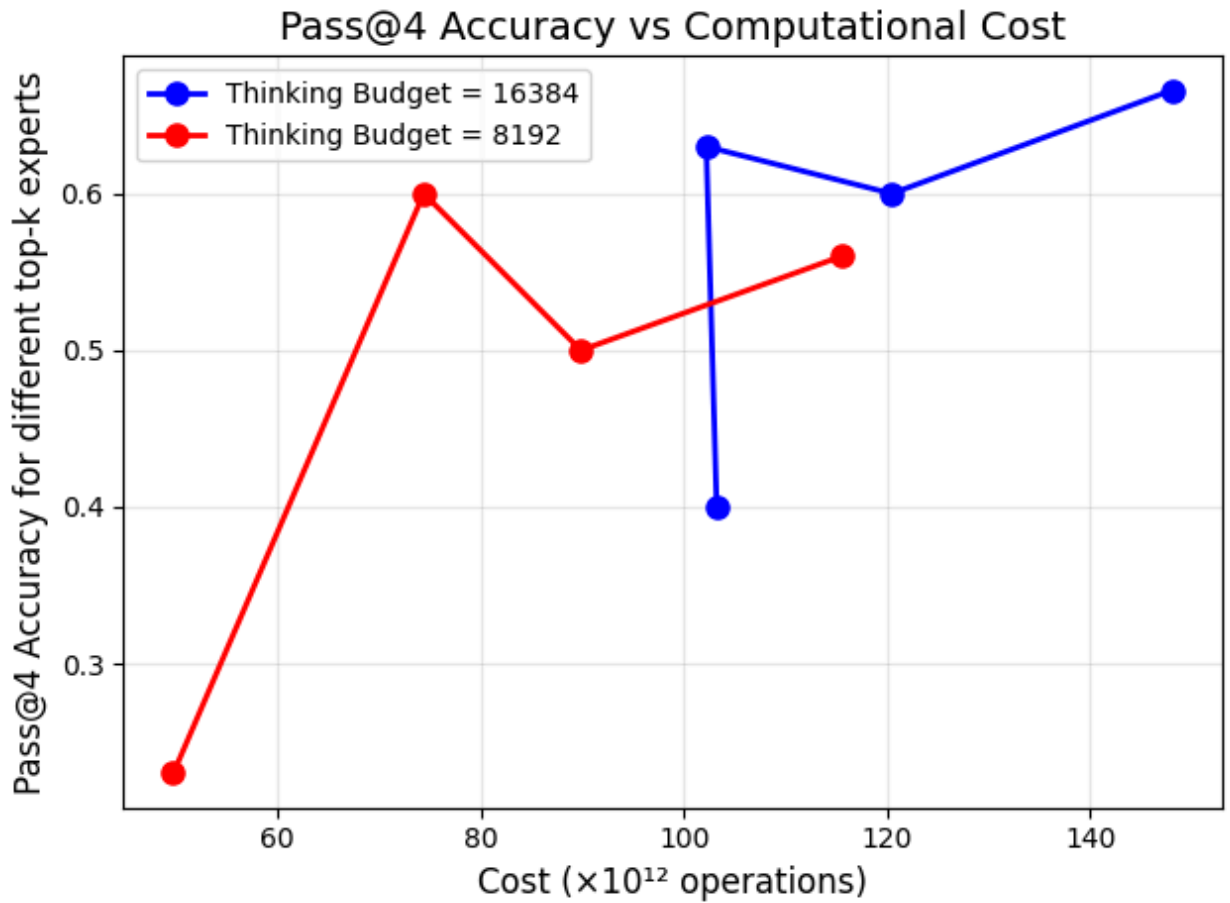
top_k: 4, pass@4: 0.73, cost: 59.03

top_k: 8, pass@4: ~0.65, cost: -

top_k: 16, pass@4: 0.60, cost: 122.17

Changes: Instead of scheduling batch

2. Latency Evaluation



Training Details

Model: Qwen3-30B-A3B

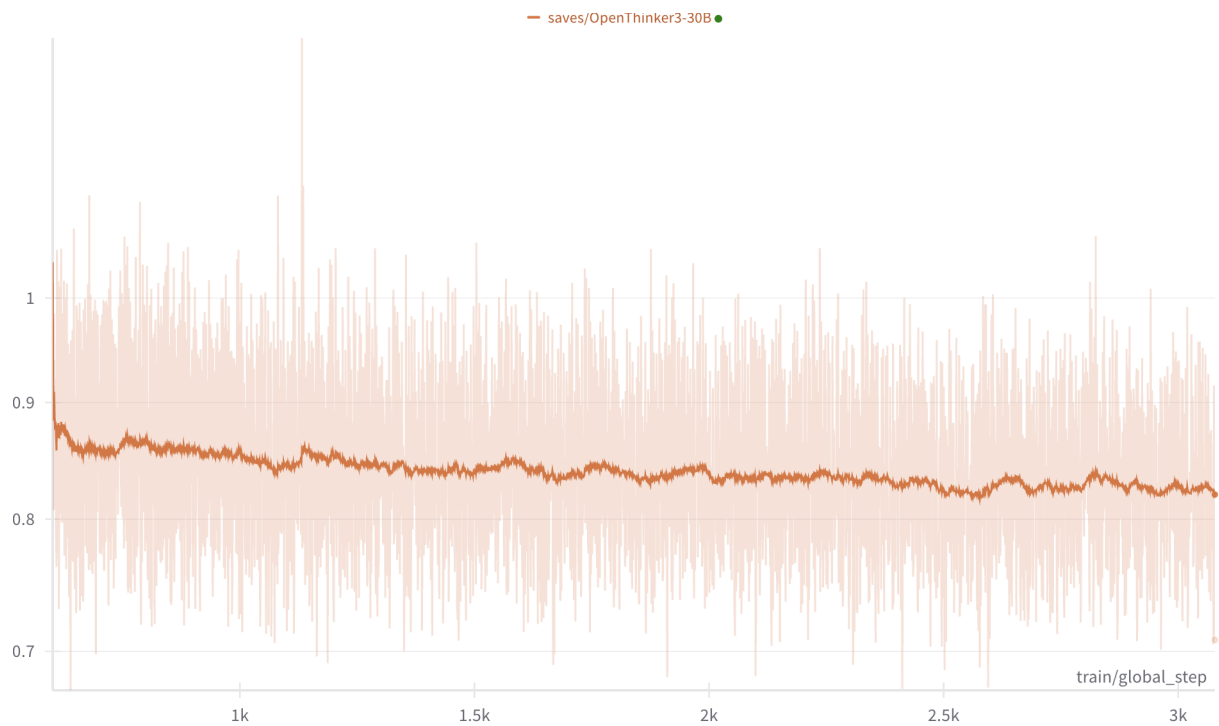
Dataset: Openthoughts3

1.2 Million datapoints with context length of 16k = ~20B tokens

Training so far: 0.8B (8 GPUs, 3k iterations, with batch size of 16)

Variation: the topk cycles through [1,2,4,8,12,16] during training with each iteration

Training curve:



Observations:

- The spikes are due to changing Ks across iterations
- The mean loss drops quickly at start and then moves really slowly probably because the model has already been trained for 30+ T tokens and at the start it just adjusts to changing K quickly.

Evaluation

matharena.ai

Hhmt,

Math500 lv4, lv5

Scaling with topk

More experts: only used when model is hard

Thinking budget: 4k

Num Experts	Accuracy
-------------	----------

1	3.3
2	20
4	36.7
8	30
16	43.3
32	43.3

Thinking budget: 8k

Num Experts	Accuracy
1	0
2	23.3
4	56.7
8	50
16	56.7
32	60

Thinking budget: 16k

Num Experts	Accuracy
1	0
2	33.3

4	60
8	56.7
16	66.7
32	60

Train only at 8
 Focus more on increasing k [4, 8, 16, 32]
 Sample K at training time
 Baseline at 4 / 8 / 16
 Compute matched training: scaling laws

Next step could be RL

Model: Qwen3-30B-A3B (mode: non-thinking)
 Dataset: gsm8k

Num Experts	Accuracy
8 (original)	94.9
1	0.8
2	2.6
4	91.5
10	95.1
12	95.5
14	95.0
16	94.2

Dataset: Math500

Num Experts	Accuracy	NLL wrt ground truth	NLL wrt original model output
8 (original)	87.2	1.502	0.079
1	1.6	10.022	9.171
2	3.6	1.890	0.778
4	81.0	1.432	0.136
6	86.2	1.464	0.091
10	84.4	1.507	0.081
12	87.6	1.505	0.087
14	85.6	1.509	0.095
16	82.8	1.502	0.105
20	80.8	1.493	0.126

Latency Measurement

Output length	Num experts	Avg Latency
2048	8 (original)	14.52
2048	1	13.60
2048	2	13.34
2048	4	13.28
2048	6	14.17
2048	10	15.22
2048	12	15.99
2048	14	16.75
2048	16	17.24
2048	20	18.58

Replacing part of the reasoning trajectory with a weaker model.

Model family: Deepseek r1 distill

Dataset: AIME 2024

Metric: cons = self consistency / majority voting

Correct answer average = 8K - 10K — AIME 2024

1.5B

cons@64: 56.67%

7B

cons@64: 76.67%

(1.5B — 2.5K) → (7B — 2.5K) → (1.5B — 2.5K) → (7B — Complete)

cons@64: 73.33%

(1.5B — 7.5K) → (7B — Complete)

cons@64: 63.33%

@

Distillation Datamix Design

Distillation Datamix Design

A common setting for distillation is as follows:

- Logits from a strong teacher model are available. The teacher model is too expensive to serve product traffic.
- We need to train a student model via distillation
- The architectural design (num params, dense vs moe) of the student is fixed based on product constraints (time to first token, tokens/second) and infra constraints (hardware specs like HBM)

The missing piece here is the datamix - the overall token budget and mixture ratios of different subsets of data (code, stem, web etc)

One option is to use the same datamix as the teacher. The teacher datamix is likely highly optimized (since it was used to train the expensive teacher model)

However, the teacher datamix is typically designed by first assuming a compute budget. This is bounded (by the number of GPUs we have and how long we are willing to train the model). Based on the compute budget, scaling ladders are used to determine the optimal allocation between parameters and tokens (compute optimal)

Today's models are in a regime where we do not have enough unique tokens to train and compute optimal models. Consequently, data is repeated or epoched. At a high level, the optimal allocation of compute is now a function of 3 things - parameters, unique tokens and number of epochs.

The student model is typically much smaller than the teacher. Therefore, it is not bound by compute. We can train it for "as long as we want" i.e. for as many tokens as we can throw at it. However, epoching too much leads to overfitting, and there are *some* real world constraints (deadlines) so we can't actually train forever. Each dataset can have an epoching limit, above which quality starts degrading. Eg: "We can train code for 4 epochs, at 5 epochs we start to see overfitting"

In Llama-4 we found evidence that training on logits (distillation) can be more robust to epoching as compared to training on pure data (tokens). So we can epoch more, which lets us

- [A] For the same token budget, allocate the mixture ratios differently across subsets like code, stem, web (eg: if we care a lot about code, we can increase epoching - and mixture weight - on the code subset of the datamix)
- [B] Train for longer than was optimal in the data-only regime, since epoching limits are higher. (eg: maybe we can train the student for 20T tokens instead of 10T tokens)

The research project would involve studying impacts of distillation on datamix design by running pre-training scaling ladder studies. Some illustrative findings could be:

- Logits let us epoch 1.5x more on code as compared to data, leading to parameter factor gain of 2x on coding evals.
 - Eg: Lets say our student model was 5B params and datamix was 20T tokens, consisting of 4T code tokens (1T unique tokens repeated 4 times. We couldn't go higher because at 5 repeats we saw overfitting.) Logit lets us safely epoch 6 times, and we can allocate 6T tokens to code, leading to improvements on code evals. The 5B distilled model now matches code performance of a 10B model train on 20T data-only regime, implying a parameter factor gain of "2x".
- Logits let us increase the token budget by 2x leading to parameter factor gain of 1.5x on coding evals.
 - Eg: In data regime, epoching limits let us train a 5B student model for 20T tokens. With logits we can train the model for 40T logits without overfitting. The quality gains are equivalent to what a 7.5B model could achieve with 20T tokens, implying a "parameter factor" gain of 1.5x

In addition, the research project will also study how the epoching limits change with respect to the number of logits per token. I (eg top-K 512 vs 256 vs 128 etc)

<synthetic data generation for distillation>

Mechanics of Memorization

Mechanics of Memorization

One of the key desirable properties of LLMs is that they encode world knowledge. Ideally we'd want the LLM to encode as much knowledge as possible.

A somewhat overly simple but useful way to think of the *amount* of knowledge is the number of facts encoded by the model.

In the real world data distribution, there is a long tail of "facts", some facts appear many times in the data, others appear far fewer times.

Qualitatively, it's apparent that LLMs easily memorize/recall common facts like "Who was the president of the USA in 2018?" vs rare facts "Who was the president of El Salvador in 1902?"

There could be many reasons that impact the learnability of a "fact". It's quite likely that the learnability of a fact is directly proportional to the number of times the fact occurs over the training corpus. There are many more documents that talk about the president of USA in 2018 vs the president of El Salvador in 1902.

Another factor is the capacity of the model. Larger models memorize more things.

Yet another factor could be the "type" of fact. It might be easier to memorize "The number of Grand Slams in a year is X" vs "The president of USA in 2018 is Y" even if both facts appear the same number of times in the corpus. It could be because the "probable" answers to X is much smaller than the probable answers to Y.

Can we quantify the relationship between

1. learnability of a fact,
2. Number of parameters of the model
3. Number of times the fact occurs in the training corpus
4. [Bonus] Type of fact

Put another way, can we say something like the following - If a fact F is seen more than K times by a P param model, there is a $X\%$ chance that it's memorized.

Can we say something like, "The total number of facts that can be learned by a P parameter model is N " ?

Why would we want to study this?

1. It's cool! It helps us learn a bit more about LLMs

2. It could help us more optimally encode knowledge in an LLM. Eg: Lets say we find that a 7B model needs to see a fact 100 times for it to be memorized. In the natural distribution of the data some facts would be seen 1000s of times (head), while others would be seen only 10 times (tail). Since we now know that the threshold is 100, we can reduce the number of times a common fact is seen and allocate the tokens by repeating a rare fact more number of times.

Of course naively, repeating a set of documents could have negative consequences. In that case we can also study “how” to repeat documents. Eg: maybe paraphrasing them in different ways can be better than naively repeating them verbatim?

Some things to figure out:

- How to represent a fact?
- How to identify what facts are present in a particular document?
- Would it be easier to do controlled experiments if we synthesize brand new “fake” knowledge (never seen on the web) and add that to the training corpus?

Related Work

[\[2406.15720\] Scaling Laws for Fact Memorization of Large Language Models](#)

This work trains on literal “triples” of <subject, object, predicate> facts. So the setting is artificial. Our study would focus on real world natural language data. (But we can still use the <subject, object, predicate> representation to measure the facts underlying real world web documents)

Misc

Llama Idea Backlog (random order)

- From Frank Zhang Study the boundary effect of the synthetic data that are instruction-like. We probably don't want to truncate certain datasets during pre-training e.g. the instruction backtranslation dataset but there's
 1. No good way to do that in our codebase
 2. The effect is tricky to measure. Researching out both of these two will be quite interesting
 3. Eg: Lets say we are training on <instruction, response> pairs in pre-training. Given that we cut and pack sequences, a given example for training might be closer to <instruction, partial_response> (because the rest of it got cut off)
- Synthetic Data in Distillation
 - What are the scaling trends with *model generated* synthetic data?
 - Do we see consistent gains going from 1B tokens $\Rightarrow$ to 10B tokens $\Rightarrow$ 100B ? How far can we push this? How would we scale the prompts?
 - Study student-teacher gap. Eg: if we want to build a 7B model, should we distill from 70B or 405B? What are the quality-cost tradeoffs? (eg: 1 generation from 405B = 6 generations from 70B)
- Synthetic Code Data ?
- Predicting the full loss curve
 - Expand on <https://arxiv.org/abs/2408.11029> to make it useful for LLama ([current status](#))
 - Cons: Unclear what the publishing angle will be
- Loss based data selection / curation.
- CPT vs Mid-Train vs from-scratch training - when to do what and for how long?
 - I think Sheng had originally suggested this area

Questions

Info on routing - how much time is spent on routing in MoEs

Synthetic Data For Code

Synthetic Data For Code

Code is one of the more promising directions to scale synthetic data to pre-training sized datasets (> 100B tokens). Code, relative to other domains, offers ways to “verify” goodness (does it run, does it compile, does it pass test cases)

So far we have only explored code completion in Llama-4 (<prompt, correct code>), however there’s many axes along which can experiment with code

- Debug Traces - <prompt, incorrect_code, error_message, fixed_code>
- Test Case Generation - How to synthesize test cases automatically (useful for RL)

Questions to study include

- How can we synthesize >100M unique and diverse prompts?
- How can we create lever for “hardness” - we want train models on difficult problems rather than easy problems
- How do we verify correctness of model generated code - can we reliably generate synthetic test cases? How do we test the test cases for coverage and correctness?

Tab 5

Reinforcement Learning with Likelihood based Rewards

Tab 7

Base: b9e9c807327781e39769dea6faf9f116e5a66a61

Diff: <https://www.internalfb.com/diff/D78514166>